



[Home](#)

[About](#)

[Contact](#)

Sales and Revenue

PIZZA HUB

● DATA ANALYSIS USING SQL



[Home](#)[About](#)[Contact](#)

PROJECT DESCRIPTION

PIZZAHUB SALES AND REVENUE ANALYSIS USING SQL

Performed end-to-end sales data analysis on PizzaHub's database using SQL. Solved business-focused queries ranging from basic sales metrics to advanced revenue analysis. Extracted insights on order trends, customer preferences, pizza category performance, revenue contribution, peak ordering hours, and cumulative sales growth using joins, aggregations, subqueries, CTEs, and window functions.



[Home](#)[About](#)[Contact](#)

PIZZAHUB SALES AND REVENUE ANALYSIS USING SQL

Analyzed PizzaHub sales data using SQL to uncover insights into sales performance, revenue generation, customer ordering patterns, and product popularity. Performed data analysis to identify top-selling pizzas, category-wise sales distribution, peak ordering hours, revenue contribution, and cumulative revenue trends. Utilized SQL operations including `SELECT`, `FROM`, `JOIN`, `GROUP BY`, `ORDER BY`, `SUM()`, `COUNT()`, `ROUND()`, `OVER()`, `RANK()`, `LIMIT`, and subqueries to solve business-oriented analytical problems.

Tools: MySQL, SQL

Skills: Data Analysis, Joins, Aggregate Functions, Window Functions, Ranking Functions, Business Intelligence Reporting.





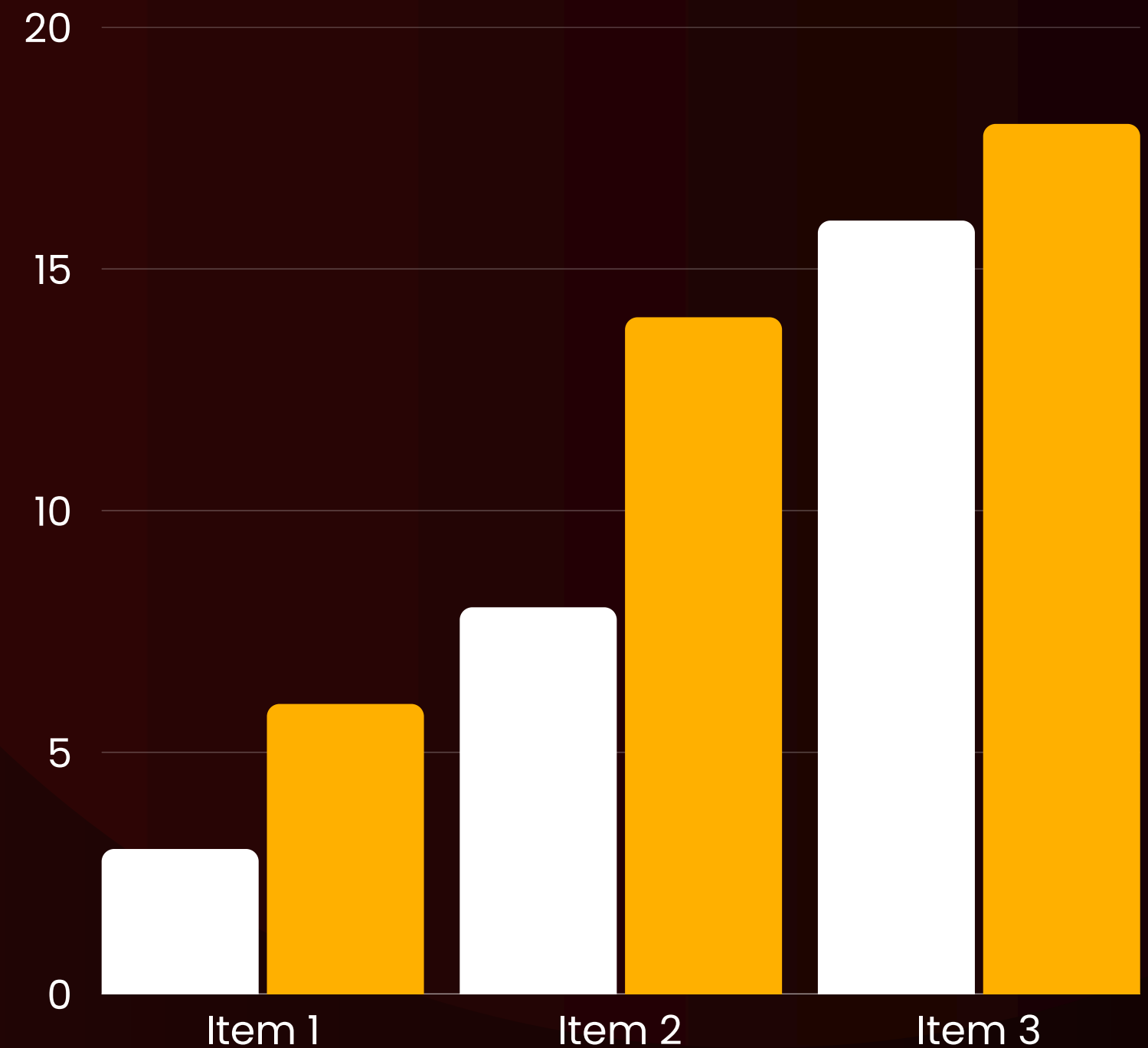
SALES REPORT

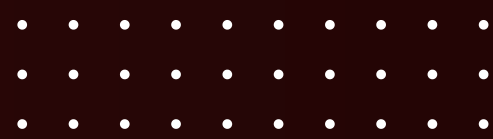
ANALYSIS

This report presents an analysis of PizzaHub sales data using SQL. The project focuses on evaluating sales performance, revenue generation, customer ordering behavior, and product popularity. Through various SQL queries, key business insights were extracted, including top-selling pizzas, category-wise sales distribution, peak ordering hours, and revenue trends. The findings demonstrate how data-driven analysis can support better business decisions and improve operational performance.

[Home](#)[About](#)[Contact](#)

● Series 1 ● Series 2



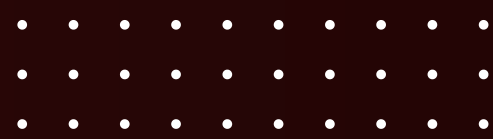
[Home](#)[About](#)[Contact](#)

RETRIEVE THE TOTAL NUMBER OF ORDERS PLACED.

```
SELECT  
    COUNT(order_id)  
FROM  
    orders;
```



Result Grid			
	COUNT(order_id)		
▶	21350		

[Home](#)[About](#)[Contact](#)

CALCULATE THE TOTAL REVENUE GENERATED FROM PIZZA SALES.

SELECT



```
ROUND(SUM(orders_details.quantity * pizzas.price),  
      2) AS total_sales
```

FROM

```
orders_details
```

JOIN

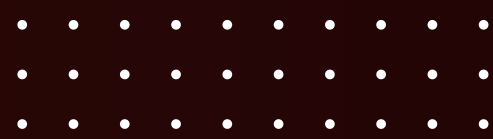
```
pizzas ON pizzas.pizza_id = orders_details.pizza_id;
```



Result Grid



	total_sales
▶	817860.05

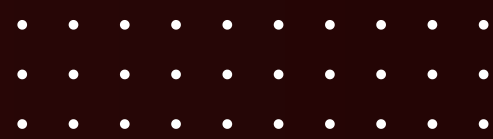
[Home](#)[About](#)[Contact](#)

IDENTIFY THE HIGHEST-PRICED PIZZA.

```
SELECT
    pizza_types.name, pizzas.price AS highest_priced
FROM
    pizzas
    JOIN
    pizza_types ON pizzas.pizza_type_id = pizza_types.pizza_type_id
ORDER BY pizzas.price DESC
LIMIT 1;
```



Result Grid			Filter Rows:	
	name	highest_priced		
▶	The Greek Pizza	35.95		

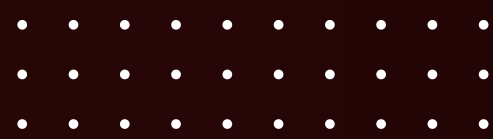
[Home](#)[About](#)[Contact](#)

IDENTIFY THE MOST COMMON PIZZA SIZE ORDERED.

```
SELECT
    pizzas.size, COUNT(orders_details.order_details_id) AS order_size
FROM
    pizzas
    JOIN
    orders_details ON pizzas.pizza_id = orders_details.pizza_id
GROUP BY pizzas.size
ORDER BY order_size DESC;
```



Result Grid		
	size	order_size
▶	L	18526
	M	15385
	S	14137
	XL	544
	XXL	28

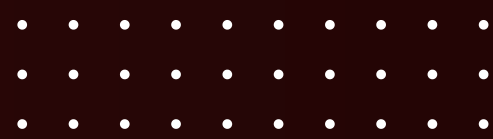
[Home](#)[About](#)[Contact](#)

LIST THE TOP 5 MOST ORDERED PIZZA TYPES ALONG WITH THEIR QUANTITY

```
SELECT
    pizza_types.name, SUM(orders_details.quantity) AS quantity
FROM
    pizzas
    JOIN
    pizza_types ON pizzas.pizza_type_id = pizza_types.pizza_type_id
    JOIN
    orders_details ON pizzas.pizza_id = orders_details.pizza_id
GROUP BY pizza_types.name
ORDER BY quantity DESC
LIMIT 5;
```



Result Grid			Filter Rows:
	name	quantity	
▶	The Classic Deluxe Pizza	2453	
	The Barbecue Chicken Pizza	2432	
	The Hawaiian Pizza	2422	
	The Pepperoni Pizza	2418	
	The Thai Chicken Pizza	2371	

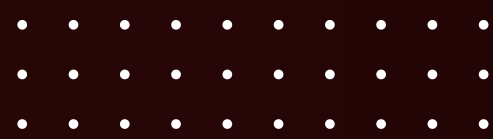
[Home](#)[About](#)[Contact](#)

JOIN THE NECESSARY TABLES TO FIND THE TOTAL QUANTITY OF EACH PIZZA CATEGORY ORDERED.

```
SELECT
    pizza_types.category,
    SUM(orders_details.quantity) AS total_category
FROM
    pizzas
    JOIN
    pizza_types ON pizzas.pizza_type_id = pizza_types.pizza_type_id
    JOIN
    orders_details ON pizzas.pizza_id = orders_details.pizza_id
GROUP BY pizza_types.category
ORDER BY total_category desc;
```



Result Grid			Filter Rows
	category	total_category	
▶	Classic	14888	
	Supreme	11987	
	Veggie	11649	
	Chicken	11050	

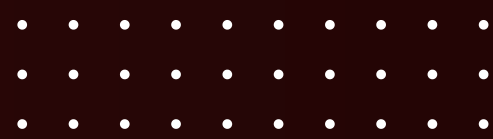
[Home](#)[About](#)[Contact](#)

DETERMINE THE DISTRIBUTION OF ORDERS BY HOUR OF THE DAY.

```
SELECT
    HOUR(order_time) AS hours, COUNT(order_id) AS counts
FROM
    orders
GROUP BY hours;
```



	hours	counts
▶	11	1231
	12	2520
	13	2455
	14	1472
	15	1468
	16	1920
	17	2336
	18	2399
	19	2009
	20	1642
	21	1198
	22	663
	23	28
	10	8
	9	1

[Home](#)[About](#)[Contact](#)

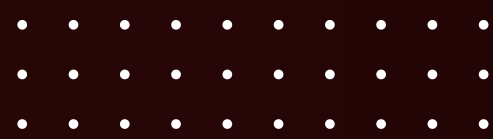
JOIN RELEVANT TABLES TO FIND THE CATEGORY-WISE DISTRIBUTION OF PIZZAS

```
SELECT  
    category, COUNT(name) AS cat_distr  
FROM  
    pizza_types  
GROUP BY category;
```



Result Grid			Filter
	category	cat_distr	
▶	Chicken	6	
	Classic	8	
	Supreme	9	
	Veggie	9	

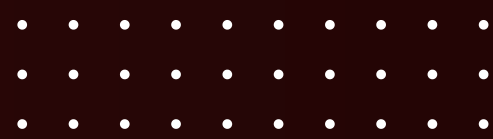


[Home](#)[About](#)[Contact](#)

GROUP THE ORDERS BY DATE AND CALCULATE THE AVERAGE NUMBER OF PIZZAS ORDERED PER DAY.

```
SELECT
    ROUND(AVG(no_of_order), 0) AS avg_piz_per_day
FROM
    (SELECT
        orders.order_date AS order_day,
        SUM(orders_details.quantity) AS no_of_order
    FROM
        orders
    JOIN orders_details ON orders.order_id = orders_details.order_id
    GROUP BY orders.order_date) AS order_num;
```

Result Grid			
	avg_piz_per_day		
▶	138		

[Home](#)[About](#)[Contact](#)

DETERMINE THE TOP 3 MOST ORDERED PIZZA TYPES BASED ON REVENUE.

```
SELECT
    pizza_types.name,
    SUM(pizzas.price * orders_details.quantity) AS rev
FROM
    orders_details
    JOIN
    pizzas ON orders_details.pizza_id = pizzas.pizza_id
    JOIN
    pizza_types ON pizza_types.pizza_type_id = pizzas.pizza_type_id
GROUP BY pizza_types.name
ORDER BY rev DESC
LIMIT 3;
```



Result Grid				Filter Rows:
	name	rev		
▶	The Thai Chicken Pizza	43434.25		
	The Barbecue Chicken Pizza	42768		
	The California Chicken Pizza	41409.5		

[Home](#)[About](#)[Contact](#)

CALCULATE THE PERCENTAGE CONTRIBUTION OF EACH PIZZA TYPE TO TOTAL REVENUE.

```
SELECT
    pizza_types.category,
    ROUND((SUM(pizzas.price * orders_details.quantity) / (SELECT
        ROUND(SUM(orders_details.quantity * pizzas.price),
            2) AS total_sales
    FROM
        orders_details
        JOIN
        pizzas ON pizzas.pizza_id = orders_details.pizza_id)) * 100,
    2) AS tot_rev
FROM
    orders_details
    JOIN
    pizzas ON orders_details.pizza_id = pizzas.pizza_id
    JOIN
    pizza_types ON pizza_types.pizza_type_id = pizzas.pizza_type_id
GROUP BY pizza_types.category
ORDER BY tot_rev DESC;
```

Result Grid			Filter
	category	tot_rev	
▶	Classic	26.91	
	Supreme	25.46	
	Chicken	23.96	
	Veggie	23.68	



[Home](#)[About](#)[Contact](#)

ANALYZE THE CUMULATIVE REVENUE GENERATED OVER TIME.

```
select order_date,  
       sum(revn) over(order by order_date) as rev_cumm  
from  
  (select orders.order_date,  
         sum(orders_details.quantity* pizzas.price)  
         as revn  
   from orders join orders_details on  
         orders.order_id = orders_details.order_id  
         join pizzas on  
         pizzas.pizza_id = orders_details.pizza_id  
   group by orders.order_date)  
as sales;
```

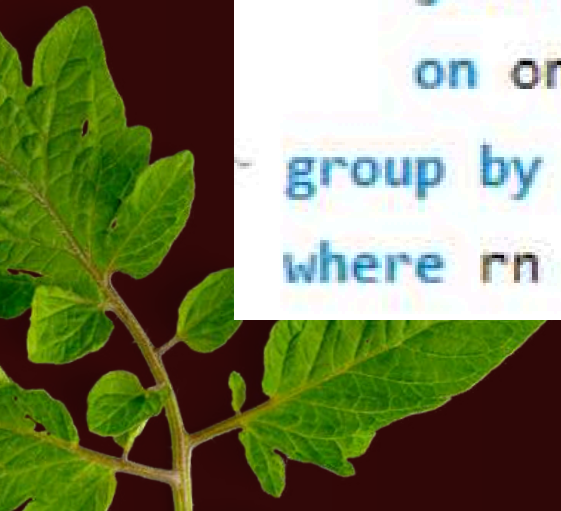
Result Grid		Filter Rows:
	order_date	rev_cumm
	2015-01-11	25862.65
	2015-01-12	27781.7
	2015-01-13	29831.3000000000003
	2015-01-14	32358.7000000000004
	2015-01-15	34343.500000000001
	2015-01-16	36937.650000000001
	2015-01-17	39001.750000000001
	2015-01-18	40978.6000000000006
	2015-01-19	43365.750000000001
	2015-01-20	45763.650000000001
	2015-01-21	47804.200000000001
	2015-01-22	50300.900000000001
	2015-01-23	52724.6000000000006
	2015-01-24	55013.8500000000006
	2015-01-25	56631.400000000001

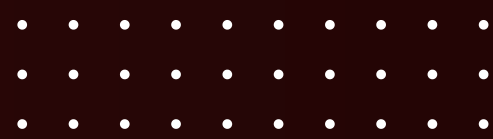
[Home](#)[About](#)[Contact](#)

DETERMINE THE TOP 3 MOST ORDERED PIZZA TYPES BASED ON REVENUE FOR EACH PIZZA CATEGORY.

```
select name, revenue from
  (select category,name , revenue,
    rank() over(partition by category order by revenue desc) as rn
  from
    (select pizza_types.category, pizza_types.name,
      sum(orders_details.quantity * pizzas.price) as revenue
    from pizza_types join pizzas
      on pizza_types.pizza_type_id = pizzas.pizza_type_id
    join orders_details
      on orders_details.pizza_id = pizzas.pizza_id
    group by pizza_types.category, pizza_types.name ) as a)as b
where rn <= 3;
```

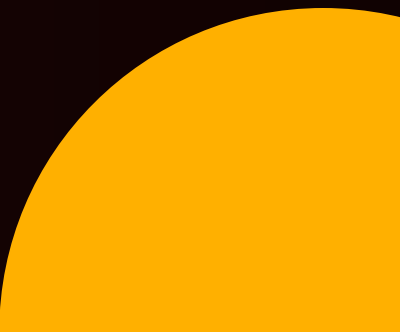
Result Grid   Filter Rows:		
	name	revenue
▶	The Thai Chicken Pizza	43434.25
	The Barbecue Chicken Pizza	42768
	The California Chicken Pizza	41409.5
	The Classic Deluxe Pizza	38180.5
	The Hawaiian Pizza	32273.25
	The Pepperoni Pizza	30161.75
	The Spicy Italian Pizza	34831.25
	The Italian Supreme Pizza	33476.75
	The Sicilian Pizza	30940.5
	The Four Cheese Pizza	32265.700000000065
	The Mexicana Pizza	26780.75
	The Five Cheese Pizza	26066.5



[Home](#)[About](#)[Contact](#)

CONCLUSION

- Successfully analyzed PizzaHub sales data using SQL.
- Identified key trends in customer orders, revenue generation, and product performance.
- Determined top-selling pizzas, popular categories, and peak ordering periods.
- Applied SQL techniques such as joins, aggregations, subqueries, and window functions to extract meaningful business insights.
- Demonstrated how data analysis can support informed business decisions and improve overall sales performance.





[Home](#)

[About](#)

[Contact](#)

THANK YOU

SHUBHANGI SONKAR

- IIT DELHI | PRODUCTION & INDUSTRIAL ENGINEERING